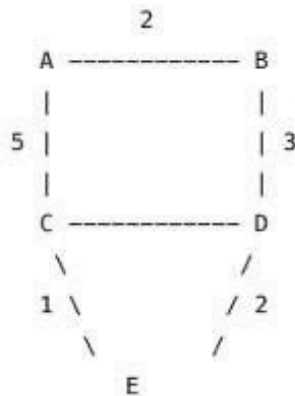


Exercise 2: Link State and Distance Vector Routing

Name: Phubes Sirimaytawut Student ID: 6510450810 Date: 21/09/2026

Learning objectives: Apply Dijkstra and Bellman-Ford; construct forwarding tables; analyze convergence, failures, and count-to-infinity.

Network topology:



Part 1 — Link State Routing

Question 1 — Dijkstra's Algorithm

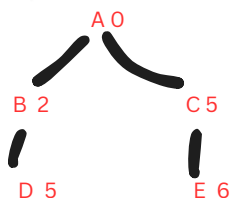
Starting at router A, execute Dijkstra's algorithm. Show every update and the node confirmed at each step.

Step	Confirmed Node	D(B), Prev.	D(C), Prev.	D(D), Prev.	D(E), Prev.
0	A	2, A	5, A	inf, -	inf, -
1	B	done	5, A	5, B	inf, -
2	C	done	done	5, B	6, C
3	D	done	done	done	6, C
4	E	done	done	done	done

Question 2 — Shortest-Path Tree

Draw the shortest-path tree rooted at A, then list the shortest path and total cost.

SPT sketch:



Destination	Shortest Path	Total Cost
B	A → B	2
C	A → C	5
D	A → B → D	5
E	A → C → E	6

A-B=2, A-C=5, B-D=3, C-D=1, C-E=1, D-E=2

1 เลือก B เพราะมี cost ต่ำสุดคือ 2 แล้ว relax neighbor ของ B ก็จะได้ D = 2+3 = 5

2 เลือก C เพราะ C กับ D ค่าเท่ากันที่ 5 แต่เลือก C ก่อนเพราะมาก่อนตามตัวอักษร แล้ว relax neighbor ของ C ก็จะได้ D = 5+1 = 6 ซึ่งแพงกว่า 5 เลยไม่อัปเดต แต่ E = 5+1 = 6 เลยอัปเดต

3 เลือก D ค่า 5 แล้ว relax neighbor ก็จะได้ E = 5+2 = 7 ซึ่งแพงกว่า 6 เลยไม่อัปเดต

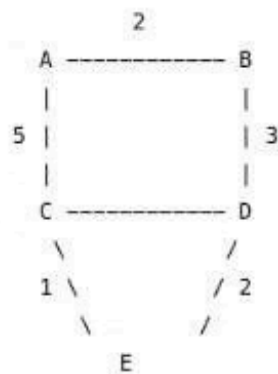
4 เลือก E ค่า 6 เสร็จสิ้น

Question 3 – Forwarding Table

Destination	Shortest Cost	Next Hop	Complete Path
B	2	B	A → B
C	5	C	A → C
D	5	B	A → B → D
E	6	C	A → C → E

Part 2 – Distance Vector Routing

Consider the same network. Initially, each router knows only the cost to its directly connected neighbors.



Question 4 – Bellman-Ford Update

Router A receives the following distance vectors from its neighbors:

From B:

A=2, B=0, C=4, D=3, E=5;

From C:

A=5, B=4, C=0, D=1, E=1.

Use the Bellman-Ford equation. Calculate router A's new distance to:

Show all calculations, including the neighbor through which the minimum-cost route is obtained.

Destination	Via B calculation	Via C calculation	Minimum Cost	Next Hop
B	2+0=2	5+4=9	2	B
C	2+4=6	5+0=5	5	C
D	2+3=5	5+1=6	5	B
E	2+5=7	5+1=6	6	C

Question 5 – Distance Vector Table

After receiving the distance vectors, construct router A's updated routing table.

Destination	Next Hop	Distance	Full Path
B	B	2	A → B
C	C	5	A → C
D	B	5	A → B → D
E	C	6	A → C → E

สูตรที่ใช้คือ $D(A \rightarrow X) = \min [\text{cost}(A \rightarrow N) + D(N \rightarrow X)]$ โดย N คือ neighbor ของ A

A มี neighbor สองตัวคือ B และ C โดย $\text{cost}(A \rightarrow B) = 2$ และ $\text{cost}(A \rightarrow C) = 5$

Distance vector ที่ได้รับมาคือ

จาก B A=2, B=0, C=4, D=3, E=5

จาก C A=5, B=4, C=0, D=1, E=1

คำนวณแต่ละปลายทาง

ไป B ผ่าน B: $2+0=2$, ผ่าน C $5+4=9$ เลือก 2 ไปทาง B

ไป C ผ่าน B: $2+4=6$, ผ่าน C $5+0=5$ เลือก 5 ไปทาง C

ไป D ผ่าน B: $2+3=5$, ผ่าน C $5+1=6$ เลือก 5 ไปทาง B

ไป E ผ่าน B: $2+5=7$, ผ่าน C $5+1=6$ เลือก 6 ไปทาง C

Part 3 — Routing Convergence

Question 6 — Link Failure

The C-D link fails. Explain separately what happens with Link State and Distance Vector routing.

Link State routing.

Your answer should describe:

- Which router(s) detect the failure.
- What information is flooded through the network.
- How routers update their link-state databases.
- How routers calculate new shortest paths.

C และ D ตรวจพบว่า link ระหว่างกันขาดกันเพราะหยุดรับ signal จากกัน จากนั้น C และ D จะส่ง LSA ใหม่ flood ไปทั่วทั้ง network เพื่อแจ้งให้ทุก router รู้ว่า link นี้ขาดแล้ว ทุก router ที่ได้รับ LSA จะอัปเดต link-state database ของตัวเองแล้วรัน Dijkstra ใหม่เพื่อคำนวณเส้นทางที่สั้นที่สุด การ converge จึงเกิดขึ้นเร็วและไม่มี loop

Distance Vector routing.

Your answer should describe:

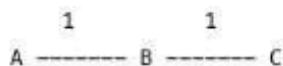
- How routers learn about the failure.
- How updated distance vectors propagate.
- Why routers may temporarily have incorrect routing information.
- How routing eventually converges.

C และ D ตรวจพบว่า link ขาดเช่นกัน แต่จะอัปเดตด้วยการส่ง distance vector ใหม่ไปให้ neighbor ของตัวเองเท่านั้น การอัปเดตจึงค่อยๆ กระจายออกไปทีละ hop ในช่วงนี้ router ที่ยังไม่ได้รับข้อมูลใหม่จะยังคงส่งแพ็กเก็ตผ่าน link ที่ขาดอยู่ ทำให้เกิด routing loop ชั่วคราวได้ การ converge จึงช้ากว่า Link State มาก

Link State	Distance Vector
C และ D ตรวจพบ link failure กันที	C และ D ตรวจพบ link failure กันที
Flood LSA ไปทั่ว network	ส่ง distance vector ใหม่ให้ neighbor เท่านั้น
ทุก router อัปเดต LSDB แล้วรัน Dijkstra ใหม่	การอัปเดตกระจายทีละ hop ช้ากว่ามาก
Converge เร็ว ไม่มี loop	เกิด loop ชั่วคราวระหว่างรอ converge

Part 4 — Count-to-Infinity

Consider



Initially A reaches C through B at cost 2. The B-C link fails, but B receives stale information from A saying C is reachable at cost 2. โทโพโลยีที่ใช้คือ A-1-B-1-C โดย A ถึง C ผ่าน B ด้วย cost 2

Question 7

1. Why can this create a routing loop? link B-C ขาด B ไม่สามารถถึง C ได้แล้ว แต่ B เห็นว่า A บอกว่าถึง C ได้ด้วย cost 2 จึงคิดว่าไปหา C ได้ผ่าน A ด้วย cost 1+2=3 แต่จริงๆ แล้ว A ถึง C ผ่าน B อยู่ดี ทำให้เกิดวนระหว่าง A กับ B ไปเรื่อยๆ
2. Show how the estimated distance to C can increase over successive iterations.
3. What is count-to-infinity? ปัญหาที่ router ใน Distance Vector routing ต่างเพิ่ม cost ไปยัง destination ที่ไม่สามารถถึงได้ขึ้นเรื่อยๆ ไม่หยุด เพราะต่างรับข้อมูลเก่าจากกันและกันวนไปเรื่อยๆ จนกว่าจะถึงค่า infinity ที่กำหนดไว้ เช่น 16 ใน RIP
4. Why does it occur in Distance Vector routing? router แต่ละตัวรู้แค่ cost ที่ neighbor บอกมา ไม่รู้ topology จริงๆ ของ network ทั้งหมด เลยไม่สามารถรู้ได้ว่าเส้นทางที่ neighbor บอกนั้นวนกลับมาผ่านตัวเองหรือไม่
5. Name a mitigation approach. ใช้ Split Horizon คือ router จะไม่ส่ง route กลับไปให้ neighbor ที่เป็นคนบอก route นั้นมาตั้งแต่แรก ทำให้ B ไม่ได้รับข้อมูลว่า A ถึง C ได้ เพราะ A เรียนรู้เส้นทางนั้นมาจาก B อยู่แล้ว

Iteration	B คิดว่าถึง C	A คิดว่าถึง C
0 (ก่อน failure)	1	2
1 (B-C ขาด)	3 (ผ่าน A ใช้ค่าเก่าของ A=2)	2
2	3	4 (ผ่าน B ใช้ค่าใหม่ของ B=3)
3	5 (ผ่าน A ใช้ค่าใหม่ของ A=4)	4
4	5	6
...	เพิ่มขึ้นเรื่อยๆ	เพิ่มขึ้นเรื่อยๆ

Part 5 — Compare LS and DV

Question 8

Characteristic	Link State	Distance Vector
Basic algorithm	Dijkstra	Bellman-Ford
Information maintained	เก็บ topology map ของทั้ง network	เก็บแค่ distance ไปปลายทางและ next hop
Information exchanged	ส่ง LSA flood ไปด้วย network	ส่ง distance vector ให้เฉพาะ neighbor
Algorithm	router รับ Dijkstra บน LSDB อีละ	router ใช้ข้อมูลจาก neighbor เท่านั้น
Topology knowledge	รู้ topology ทั้งหมด	รู้แค่ที่ neighbor บอกมา
Routing updates	ส่งเมื่อ topology flood ไปด้วย network	ส่งเป็นรอบๆ ตามเวลา
Convergence	เร็ว	ช้า
Routing loops	ไม่เกิด	อาจเกิดได้ระหว่าง converge
Count-to-infinity	ไม่เกิด	อาจเกิดได้
Example protocol	OSPF, IS-IS	RIP

Part 6 — Design Question

Question 9

You are designing a routing system for an enterprise with 500 routers requiring fast convergence, accurate topology knowledge, efficient routing, and hierarchical design. Choose LS or DV and justify your choice with at least three technical arguments. Your answer should discuss at least three advantages and/or disadvantages of your chosen approach.

เลือก Link State (OSPF) สำหรับ enterprise 500 routers

เหตุผล 1 Converge เร็วกว่ามาก

ใน network ขนาด 500 routers ถ้า link ขาดขึ้นมา ต้องการให้ทุก router รู้และปรับเส้นทางได้เร็วที่สุด Link State flood ข้อมูลไปถึงทุก router ได้ภายในไม่กี่วินาที แต่ Distance Vector ต้องรอให้ข้อมูลกระจายทีละ hop ซึ่งใน network 500 ตัวอาจใช้เวลานานมาก

เหตุผล 2 ไม่เกิด routing loop และ count-to-infinity

เพราะทุก router รู้ topology จริงๆ ของทั้ง network แล้วคำนวณเส้นทางด้วยตัวเอง จึงไม่มีทางเกิด loop หรือ count-to-infinity ซึ่งสำคัญมากใน enterprise ที่ต้องการความเสถียร

เหตุผล 3 รองรับ hierarchical design ด้วย OSPF Areas

OSPF แบ่ง network เป็น area ได้ ทำให้ LSA flood อยู่แค่ใน area ของตัวเองไม่ต้องกระจายไปถึง 500 routers ช่วยลด overhead และทำให้ scale ได้ดีขึ้นมาก

Challenge — Algorithmic Analysis

Question 10

For N routers and E links, compare LS and DV in information maintained, communication overhead, computation, convergence, and scalability. Explain why large-scale intra-domain routing generally favors Link State.

Factor	Link State	Distance Vector
Information maintained	$O(N+E)$ ต่อ router เก็บ LSDB ทั้งหมด	$O(N)$ ต่อ router เก็บแค่ distance table
Communication overhead	$O(N \times E)$ flood LSA ทุก router	$O(N^2)$ ส่ง vector ให้ neighbor ทุกรอบ
Computation	$O(E \log N)$ รับ Dijkstra	$O(N)$ รับ Bellman-Ford
Convergenceเร็ว	$O(\text{diameter})$ ช้า	$O(N)$ รอบในกรณีแย่สุด
Scalability	ดีเมื่อใช้ area hierarchy	แย่เมื่อ network ใหญ่ขึ้น

ที่ Link State เหมาะกับ large-scale intra-domain routing มากกว่าเพราะ Dijkstra ให้เส้นทางที่ถูกต้องและ loop-free เสมอ การ flood LSA ทำให้ converge เร็ว และ OSPF ยังรองรับ area hierarchy ที่ช่วยลด overhead ใน network ขนาดใหญ่ได้อีกด้วย ต่างจาก Distance Vector ที่ช้า เกิด loop ได้ง่าย และมีปัญหา count-to-infinity ซึ่งทำให้ไม่เหมาะกับ network ขนาดใหญ่เลย